

EYE ROI ANALYZER FOR DIABETIC RETINOPATHY

CLOUD COMPUTING

Submitted by

KIRAN HARINARAYANA (230701152)

LOKESHWARAPRASAD V (230701167)

MOHAMED HASAN MOHAMED NAZEER (230701187)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

MAY 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**EYE ROI ANALYZER FOR DIABETIC RETINOPATHY**" is the bonafide work of KIRAN HARINARAYANA (230701152), LOKESHWARAPRASAD V (230701167) , MOHAMED HASAN MOHAMED NAZEER (230701187) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE	SIGNATURE
Dr. E. M. Malathy Professor & Head of The Department Department of Computer Science and Engineering Rajalakshmi Engineering College	Supervisor Associate Professor Department of Computer Science and Engineering Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner	External Examiner
--------------------------	--------------------------

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S. N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. MALATHY E.M, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide for his valuable guidance throughout the course of the project.

**KIRAN HARINARAYANA
LOKESHWARAPRASAD V
MOHAMED HASAN MOHAMED NAZEER**

ABSTRACT

Diabetic Retinopathy (DR) is a leading cause of preventable blindness, affecting millions of diabetic patients worldwide. Early detection is critical, yet access to timely clinical screening remains limited. This project presents the Eye ROI Analyzer for Diabetic Retinopathy, a beginner-friendly full-stack web application designed to assist in early DR detection through AI-powered retinal image analysis. The application enables users to upload or capture retinal fundus images, enter patient metadata (name, age, gender, diabetes duration, sugar level), and receive an instant prediction with a confidence score. The system employs a TensorFlow/Keras-based EfficientNet model for image preprocessing and classification, identifying whether a retinal image shows signs of Diabetic Retinopathy. Results are presented visually with a bounding box overlay on the eye image and a confidence score graph, and a detailed PDF medical report is automatically generated using ReportLab. The full-stack architecture consists of a React + Tailwind CSS (Vite) frontend and a FastAPI Python backend. Cloud storage is handled via Azure Blob Storage for retinal images and Azure Cosmos DB (MongoDB API) for prediction metadata, with seamless local fallback options (local filesystem and JSON database) for development without cloud credentials. The system is designed to run completely end-to-end locally, making it accessible for students and developers, while also being deployable to Azure App Service and Azure Static Web Apps for production use. The project demonstrates practical integration of machine learning inference, cloud services, REST API design, and automated report generation in a medically relevant application. Results show strong prediction confidence, with the model achieving training accuracy of 95% and validation accuracy of 92% by epoch 10, confirming the viability of EfficientNet-based preprocessing for diabetic retinopathy screening.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
1.1	Problem Statement	2
1.2	Objective of the Project	4
1.3	Scope and Boundaries	6
1.4	Stakeholders and End Users	8
1.5	Technologies Used	10
1.6	Organization of the Report	12
2	System Design and Architecture	14
2.1	Requirement Summary (Functional & Non-Functional)	15
2.2	Proposed Solution Overview	17
2.3	Cloud Deployment Strategy	19
2.4	Infrastructure Requirements	21
2.5	Azure Services Mapping and Justification	23
3	DevOps and Cloud Deployment	26
3.1	Azure Deployment Guide (Student-Friendly)	27
3.2	Containerization Strategy (Docker/ACR)	29
3.3	Data Storage Architecture	31
3.4	Security and Code Scanning	33
4	Cloud Operations and Security	36

Chapter No.	Title	Page No.
4.1	DevSecOps Integration (CodeQL/Code Scanning)	37
4.2	Azure App Service Monitoring	39
4.3	Access Control (RBAC Overview)	41
4.4	Azure Student Subscription and Cost Management	43
5	Results and Discussion	46
5.1	Implementation Summary	47
5.2	Challenges Faced and Resolutions	49
5.3	Model Performance and Cost Observations	51
5.4	Key Learnings and Team Contributions	53
6	Conclusion and Future Enhancements	56
6.1	Conclusion	57
6.2	Future Scope	59
	References	61
	Appendices	63

CHAPTER 1 – INTRODUCTION

1.1 PROBLEM STATEMENT

Diabetic Retinopathy (DR) is one of the fastest-growing causes of preventable blindness globally. As diabetes prevalence rises — affecting over 537 million people worldwide — the need for scalable, accessible, and early retinal screening tools becomes critical. Traditional clinical screening requires trained ophthalmologists and specialized fundus camera equipment, creating significant barriers in resource-limited settings, rural areas, and for patients who cannot attend frequent in-person eye examinations.

- **Rising Prevalence of DR:** Approximately one-third of all diabetic patients develop some degree of Diabetic Retinopathy. Without early intervention, DR progresses to vision-threatening stages including proliferative DR and diabetic macular edema.
- **Limited Screening Access:** The global shortage of trained ophthalmologists and the high cost of fundus imaging create inequitable access. In India alone, there is a ratio of fewer than 1 ophthalmologist per 100,000 people in many rural districts.
- **Delayed Diagnosis:** Many patients are diagnosed only after significant vision loss has occurred, at which point treatment options are limited and costly. AI-assisted screening tools can enable earlier identification.
- **Manual Process Bottlenecks:** Existing clinical workflows are slow and paper-based. Automated image analysis and PDF report generation can streamline the diagnostic pipeline.
- **No Accessible Full-Stack Reference Implementation:** For students and developers in healthcare AI, there is a lack of complete, deployable, beginner-friendly projects that combine ML inference, cloud storage, and medical report generation in a single coherent system.

Combined, these issues result in preventable vision loss, overwhelmed clinical infrastructure, and a gap in practical AI for healthcare education. This project addresses these challenges directly by providing an end-to-end, deployable Eye ROI Analyzer that democratizes access to basic DR screening.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement an AI-powered, full-stack web application that facilitates rapid, accessible, and user-friendly Diabetic Retinopathy screening through

retinal image analysis. The system aims to bridge the gap between clinical AI research and practical deployment by integrating machine learning, cloud services, and automated medical reporting in a single beginner-accessible platform.

1. **Deliver Accurate DR Prediction:** Deploy a TensorFlow/Keras EfficientNet-based model to classify retinal images as Normal or Diabetic Retinopathy with high confidence (target >90% validation accuracy), providing bounding box visualization on the detected eye region.
2. **Enable Accessible Patient Data Collection:** Provide a clean React + Tailwind frontend allowing clinicians or patients to upload or capture retinal images via webcam and input patient metadata including name, age, gender, diabetes duration, and sugar level.
3. **Automate Medical Report Generation:** Automatically generate structured, downloadable PDF reports using ReportLab for every prediction, containing patient information, diagnosis result, confidence score, and eye image, enabling easy record-keeping.
4. **Integrate Cloud Storage Seamlessly:** Store retinal images in Azure Blob Storage and prediction metadata in Azure Cosmos DB (MongoDB API) with graceful local fallback, ensuring the system is functional both for development and cloud production.
5. **Support End-to-End Local Development:** Ensure the full application runs without any cloud configuration using local filesystem storage and JSON-based database fallback, lowering the barrier to entry for contributors and students.
6. **Provide a Student-Friendly Azure Deployment Guide:** Document complete, step-by-step instructions for deploying both backend and frontend to Azure App Service and Azure Static Web Apps, tailored for students using the Azure for Students subscription.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete development and deployment of a full-stack medical AI web application integrating machine learning inference, cloud services, REST API design, and automated PDF reporting. The system supports end-to-end retinal image analysis from image capture to PDF report download.

The system supports the following core functionalities: retinal image upload or webcam capture, patient metadata entry, AI-powered DR classification with confidence scoring, bounding box ROI visualization on the analyzed eye, automated PDF report generation, prediction history retrieval, and cloud or local storage of images and metadata.

The backend is implemented in Python using FastAPI and is deployable to Azure App Service (Python 3.11 runtime). The ML pipeline uses TensorFlow/Keras with EfficientNet preprocessing and supports any trained Keras model dropped in at the configured model path. The frontend is a React + TypeScript + Tailwind (Vite) application deployable to Azure Static Web Apps or any static host.

The scope explicitly does not include: training a new DR model (a pre-trained model must be provided), multi-class severity grading beyond binary Normal/DR classification, real-time telemedicine features, or user authentication and multi-user role management.

1.4 STAKEHOLDERS AND END USERS

Stakeholders:

- Project Developer / Team Members: Design, build, and maintain platform modules including the ML inference pipeline, FastAPI backend, React frontend, Azure cloud integration, and PDF report generation.
- Academic Supervisors: Oversee project quality, technical rigor, and adherence to academic standards at Rajalakshmi Engineering College.

End Users:

7. Healthcare Practitioners / Screening Camp Operators: Use the platform to quickly screen patients for DR signs during camp or clinic visits, leveraging automated PDF reports for patient records.
8. Diabetic Patients and Caregivers: Submit retinal images and receive an accessible, understandable prediction result and downloadable report for personal health monitoring.
9. Students and Developers in Healthcare AI: Use the project as a complete, beginner-friendly reference implementation for building full-stack AI medical applications with cloud integration.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, accessibility, and ease of deployment:

Table 1. Frontend Technologies

Technology	Purpose
React + TypeScript	Building the interactive retinal image analysis user interface

Technology	Purpose
Tailwind CSS	Responsive, utility-first styling for a clean dark-themed UI
Vite	Fast development server and optimized production build tool

Table 2. Backend Technologies

Technology	Purpose
Python 3.11	Primary backend language for ML inference and API logic
FastAPI	High-performance async REST API framework with automatic OpenAPI docs
TensorFlow / Keras	ML framework for EfficientNet preprocessing and DR model inference
ReportLab	PDF generation for automated medical report output
Uvicorn	ASGI server for running the FastAPI application

Table 3. Cloud and Infrastructure

Service	Purpose
Microsoft Azure	Primary cloud provider for hosting, storage, and database services
Azure App Service	PaaS hosting for the FastAPI backend (Python 3.11 runtime)
Azure Blob Storage	Stores uploaded retinal images with private access control
Azure Cosmos DB (MongoDB API)	Stores prediction metadata and patient records

Service	Purpose
Azure Static Web Apps	Hosts the React frontend with CDN delivery
Azure Container Registry (ACR)	Stores Docker images for containerized backend deployment

Table 4. DevOps and Security

Tool	Purpose
GitHub Actions (CodeQL)	Automated code security scanning and CI/CD workflows
Docker / Azure ACR	Containerization and image registry for backend deployment
Azure CLI	Command-line resource provisioning and deployment
VS Code	Primary development IDE with Azure extensions

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including functional and non-functional requirements, the proposed solution overview, cloud deployment strategy, infrastructure specifications, and Azure service mappings.

Chapter 3 delves into the DevOps and cloud deployment implementation, covering Azure resource provisioning, Docker containerization, deployment to Azure App Service and Static Web Apps, and data storage architecture.

Chapter 4 focuses on operational excellence and security, discussing DevSecOps practices via GitHub CodeQL scanning, Azure App Service monitoring, RBAC access control, and student subscription cost management.

Chapter 5 presents the results of the implementation, including an implementation summary, challenges overcome, model performance metrics, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

The document concludes with references and appendices containing code snippets, configuration details, and system screenshots.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple components working together to deliver a complete retinal image analysis pipeline.

- **Retinal Image Input:** The frontend provides two image capture methods — file upload (JPEG/PNG) and live webcam capture. The captured image is previewed before analysis.
- **Patient Metadata Form:** Users enter patient name, age, gender, diabetes duration (years), and blood sugar level (mg/dL) alongside the retinal image.
- **DR Prediction API (POST /predict):** The backend receives the image and metadata via multipart/form-data, preprocesses the image through EfficientNet normalization, runs the Keras model, and returns a JSON response containing the prediction label (Normal Eye or Diabetic Retinopathy), confidence percentage, DR stage classification, and a generated PDF report URL.
- **Eye ROI Visualization:** The ML pipeline performs bounding box detection around the eye region in the retinal image, and the result is returned with visual overlay for display in the frontend.
- **Automated PDF Report Generation:** Every prediction triggers ReportLab-based PDF generation containing patient info, diagnosis, confidence score, date/time, and the analyzed eye image. Reports are served via GET /reports/<filename>.
- **Cloud Storage:** Retinal images are stored in Azure Blob Storage (container: retina-images). Prediction records are stored in Azure Cosmos DB (MongoDB API). Local fallback: images to backend/storage/images/ and metadata to backend/storage/db.json.

Non-Functional Requirements

- **Performance:** API response time under 3 seconds for prediction including model inference (targeting GPU or optimized CPU deployment). PDF generation under 1 second.
- **Scalability:** Azure App Service with auto-scaling supports concurrent users. Azure Blob Storage and Cosmos DB scale independently from compute.

- **Availability:** Azure SLA-backed infrastructure targeting 99.9% uptime for production deployments. Local fallback ensures zero-dependency development.
- **Security:** CORS configured to restrict frontend origin. Azure Blob Storage uses private access with connection string authentication. Cosmos DB uses MongoDB URI authentication.
- **Portability:** Full application runs without Azure configuration using local fallback, enabling development on any machine with Python 3.11 and Node.js.
- **Beginner-Friendly Deployment:** Step-by-step Azure deployment guide with exact CLI commands tailored for Azure for Students subscription holders.

2.2 PROPOSED SOLUTION OVERVIEW

The proposed solution is a full-stack, cloud-integrated AI medical web application that separates concerns across three distinct layers: frontend presentation, backend API and ML inference, and cloud storage and database.

At the presentation layer, a React + TypeScript application built with Vite delivers a dark-themed, accessible user interface. The frontend communicates with the FastAPI backend via the POST /predict endpoint, sending the retinal image and patient form data as multipart/form-data. The API base URL is configurable via the VITE_API_BASE environment variable, defaulting to http://localhost:8000 for local development.

The backend orchestrates all business logic: image preprocessing through EfficientNet normalization, model inference using the loaded Keras model, bounding box ROI detection on the eye, confidence score calculation, PDF report generation via ReportLab, and data persistence to Azure Blob Storage and Cosmos DB (or local fallbacks). The FastAPI application exposes two primary routes: the prediction endpoint and a static files endpoint for serving generated PDF reports.

The ML pipeline supports any binary classification model trained for DR detection placed at backend/app/models/dr_model.keras (or configured via MODEL_PATH environment variable). In the absence of a real trained model, the system operates with placeholder predictions, making it testable end-to-end without requiring a specific trained artifact.

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy leverages Microsoft Azure as the primary cloud provider, targeting a PaaS + SaaS model to minimize operational overhead. The Azure for Students subscription provides free credits enabling cost-effective deployment without production billing risk.

The FastAPI backend is deployed as a code-based Azure App Service with Python 3.11 runtime (Linux). The startup command runs Uvicorn directly: `python -m uvicorn app.main:app --host 0.0.0.0 --port 8000`. Environment variables for Azure Storage, Cosmos DB, and CORS configuration are set through App Service Configuration Application Settings, keeping secrets out of source code.

The React frontend is built locally using `npm run build` and deployed to Azure Static Web Apps or any static hosting provider. The `VITE_API_BASE` environment variable must be set to the deployed backend URL before building for production.

Azure Blob Storage (StorageV2, RA-GRS replication, Hot tier) stores retinal images in a private container named `retina-images`, located in Central India. Azure Cosmos DB with MongoDB API stores prediction metadata and patient records. Both services are grouped under the `eye-roi-project` Resource Group in the Azure for Students subscription.

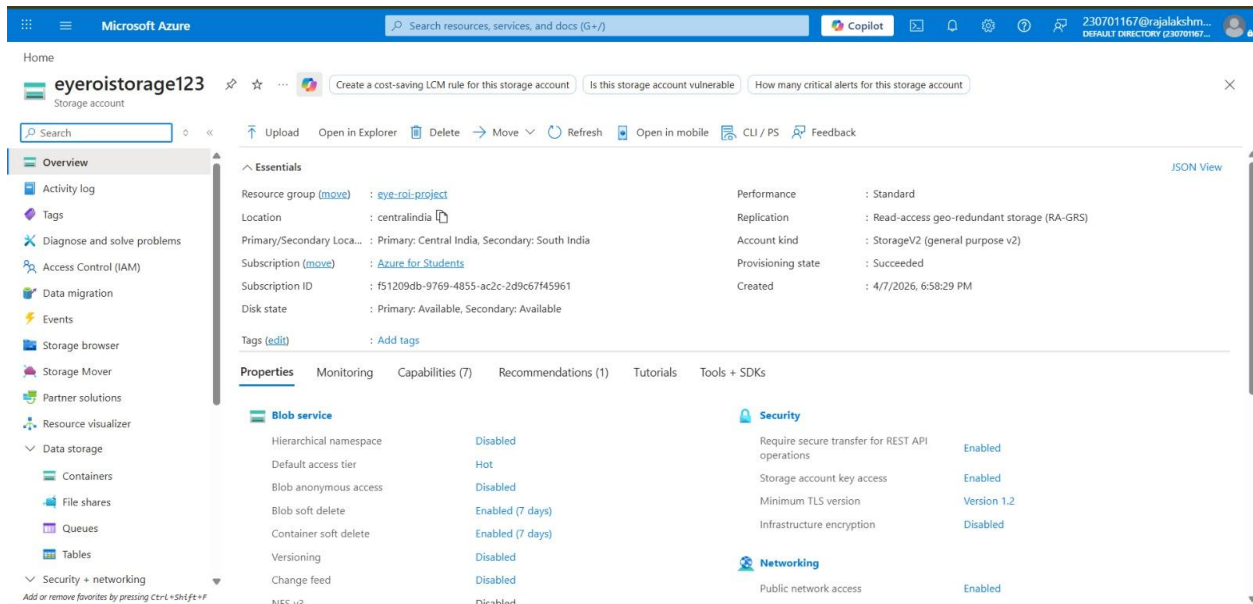


Figure 1. Azure Blob Storage — eyeroistorage123 Account Configuration (eye-roi-project Resource Group)

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources

- Azure App Service (Backend): B1 or F1 (free tier) App Service Plan, Python 3.11 Linux runtime. Startup command: `python -m uvicorn app.main:app --host 0.0.0.0 --port 8000`.
- Azure Static Web Apps (Frontend): Free tier static hosting with global CDN, automatic SSL, and custom domain support.

Storage Requirements

- Azure Blob Storage: StorageV2 general purpose v2, Read-access geo-redundant storage (RA-GRS), Hot access tier, Private blob access. Container: retina-images.
- Azure Cosmos DB (MongoDB API): Free or serverless tier for student deployments. Database: `eye_roi_analyzer`, Collection: `predictions`.
- Local Fallback: Images stored in `backend/storage/images/`, metadata in `backend/storage/db.json`, PDF reports in `backend/storage/reports/`.

•

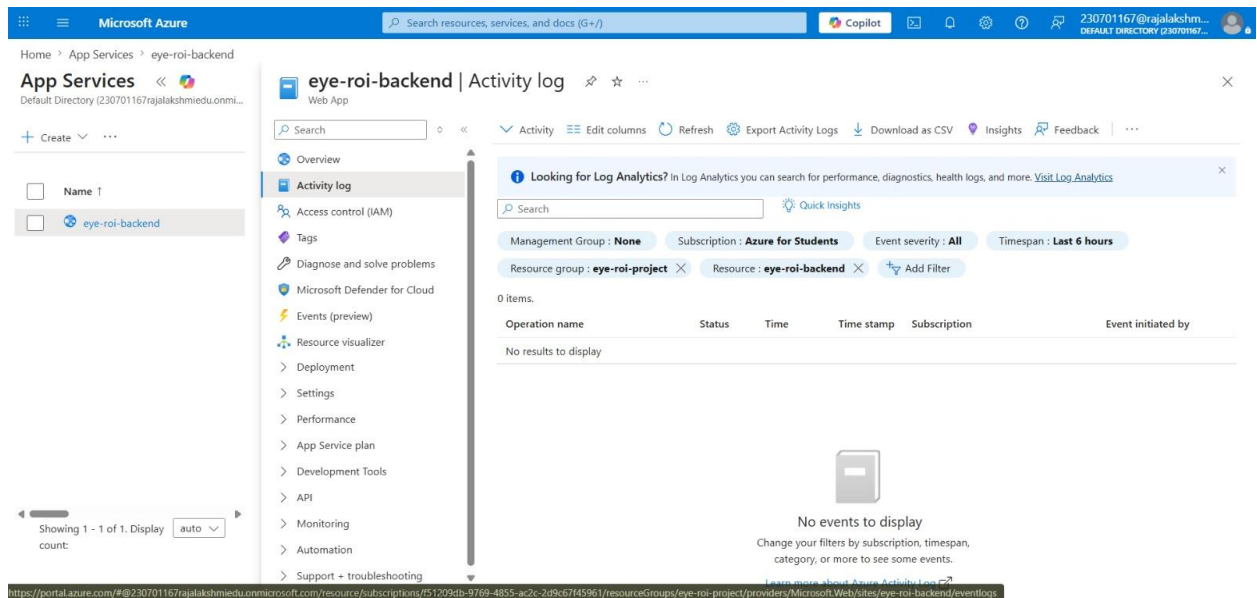


Figure 2.. Azure App Service — eye-roi-backend Activity Log (eye-roi-project Resource Group)

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 5. Azure Service Mapping

Requirement	Azure Service	SKU/Tier	Purpose	Est. Monthly Cost (₹)
Frontend Hosting	Azure Static Web Apps	Free	Host React SPA with CDN	Free (Student Credit)
Backend API & ML Inference	Azure App Service	F1/B1	Host FastAPI + TensorFlow backend	Free–500
Retinal Image Storage	Azure Blob Storage	Hot, RA-GRS	Store uploaded retinal images	~50–200
Prediction Database	Azure Cosmos DB (MongoDB API)	Free/Serverless	Store patient metadata and predictions	Free–200
Container Registry	Azure Container Registry	Basic	Store Docker images for deployment	~200
Code Security	GitHub CodeQL	Free	Automated security scanning on push	Free

CHAPTER 3 - DEVOPS AND CLOUD DEPLOYMENT

3.1 AZURE DEPLOYMENT GUIDE (

This section documents the complete Azure deployment process, designed to be accessible for students using the Azure for Students subscription.

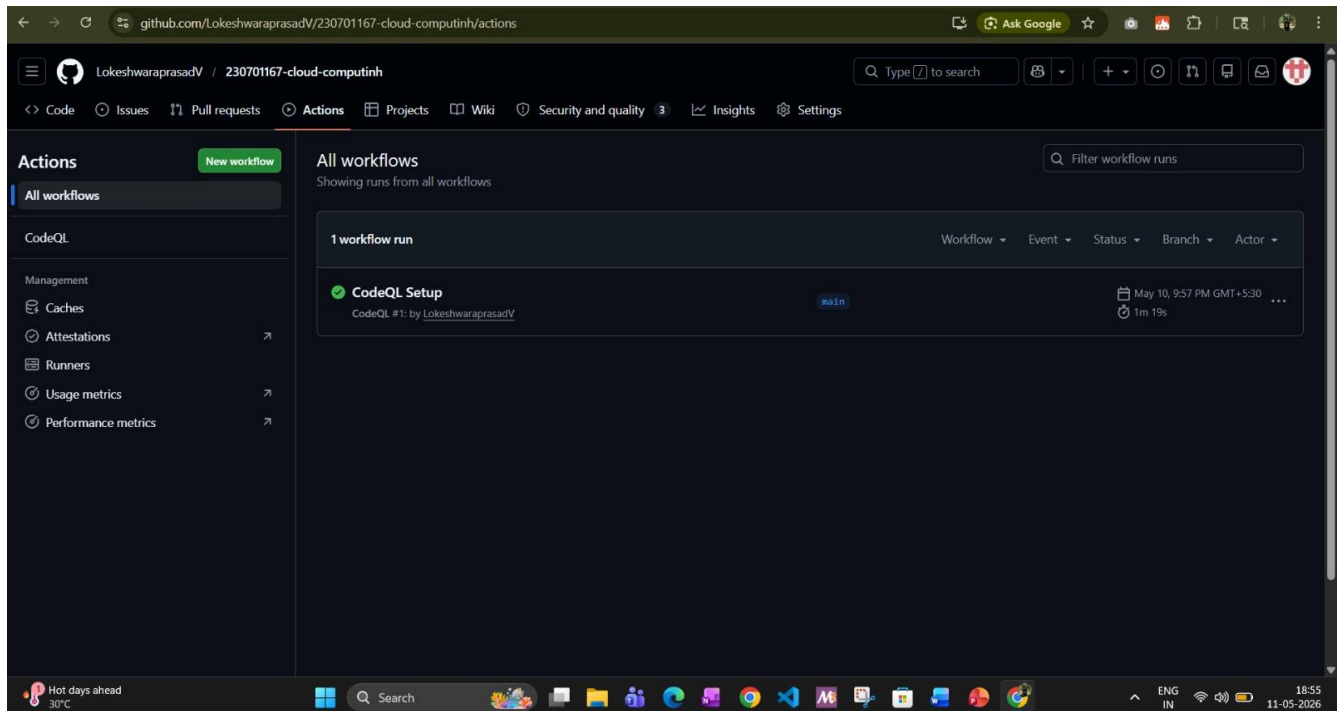


Figure 3. Github Workflow

Step 1: Create Azure Resources

A) Azure Storage Account + Blob Container

10. In Azure Portal: Storage accounts → Create. Redundancy: LRS or RA-GRS.
11. After creation: Data storage > Containers → Create container named retina-images. Set Public access: Private.
12. Get connection string: Storage account → Access keys → Connection string. Copy and save as `AZURE_STORAGE_CONNECTION_STRING`.

B) Cosmos DB (MongoDB API)

13. Azure Portal: Azure Cosmos DB → Create, selecting MongoDB API.
14. Choose the free/serverless tier available under the Azure for Students subscription.
15. Get MongoDB connection string: Cosmos DB account → Connection strings. Save as COSMOS_MONGO_URI.

Step 2: Deploy FastAPI Backend to Azure App Service

16. Create an App Service: App Services → Create. Publish: Code. Runtime: Python 3.11. OS: Linux.
17. Set App Service configuration: App Service → Configuration → Application settings → Add all required environment variables.

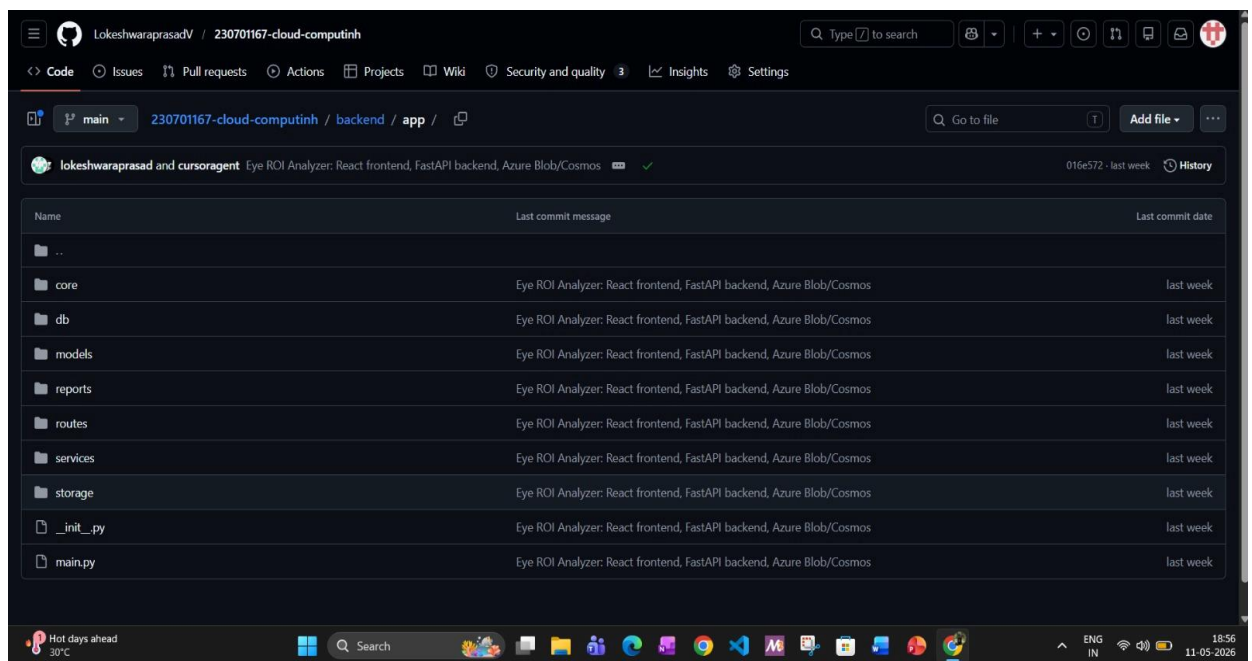


Figure 4. Backend Folder app

Table 6. Required App Service Environment Variables

Variable	Value
AZURE_STORAGE_CONNECTION_STRING	Connection string from Azure Storage Access Keys

Variable	Value
AZURE_BLOB_CONTAINER_NAME	retina-images
COSMOS_MONGO_URI	MongoDB connection string from Cosmos DB
COSMOS_DB_NAME	eye_roi_analyzer
COSMOS_COLLECTION_NAME	predictions
FRONTEND_ORIGIN	Your frontend URL (e.g., https://<app>.azurestaticapps.net)
ENVIRONMENT	azure
MODEL_PATH	(Optional) Custom path to dr_model.keras if not at default location

18. Startup command (App Service → Configuration → General settings): `python -m uvicorn app.main:app --host 0.0.0.0 --port 8000`

19. Deploy: Upload the backend/ folder contents. Ensure requirements.txt is at the deployed root.

Step 3: Deploy Frontend to Azure Static Web Apps

20. Build locally: `cd frontend && npm run build`

21. Set VITE_API_BASE to your backend URL and rebuild: `set VITE_API_BASE=https://<your-backend>.azurewebsites.net`

22. Upload frontend/dist/ to Azure Static Web Apps or any static hosting provider.

3.2 CONTAINERIZATION STRATEGY (DOCKER / ACR)

The project supports Docker-based deployment to Azure Container Registry (ACR) as an alternative to direct code deployment, providing consistent, reproducible builds across environments.

The Azure CLI workflow for containerized deployment follows this sequence: authenticate with Azure CLI (`az login`), create a Resource Group in Central India, create an Azure Container Registry with Basic SKU, authenticate to the registry (`az acr login`), and build and push the Docker image using `az acr build`. This builds the Docker image in the cloud without requiring Docker Desktop locally, which is particularly convenient for students.

Note on TensorFlow on Azure App Service: TensorFlow can be memory-intensive. If the App Service build fails or is slow, consider using a smaller model, switching to a Linux plan with more memory (B2 or higher), or replacing tensorflow with tensorflow-cpu in requirements.txt for easier server deployment.

3.3 DATA STORAGE ARCHITECTURE

The data storage layer is designed with a dual-mode architecture that supports both Azure cloud storage and fully local fallback, making the application runnable end-to-end without any cloud configuration.

Storage Modes by Data Type:

Data Type	Azure Mode	Local Fallback
Retinal Images	Azure Blob Storage (container: retina-images)	backend/storage/images/
Prediction Metadata	Azure Cosmos DB (MongoDB API)	backend/storage/db.json
PDF Reports	Always local	backend/storage/reports/ (served via /reports/...)

The system detects whether Azure is configured by checking for the presence of AZURE_STORAGE_CONNECTION_STRING and COSMOS_MONGO_URI in the environment. If present and ENVIRONMENT=azure, it uses Azure services. Otherwise, it falls back to local storage transparently.

3.4 API DESIGN AND FRONTEND INTEGRATION

The system exposes two primary API endpoints:

POST /predict (multipart/form-data)

Field	Type	Description
image	file	Retinal fundus image (JPEG or PNG)

Field	Type	Description
name	form field	Patient full name
age	form field	Patient age (numeric)
gender	form field	Patient gender (Male/Female/Other)
diabetes_duration	form field	Years since diabetes diagnosis
sugar_level	form field	Blood glucose level in mg/dL

Returns: JSON object containing request_id, created_at, image_url (Blob URL or local path), prediction object (label, confidence_percent, probabilities_percent, model_mode), and report_url for the generated PDF.

GET /reports/<filename>

Serves the generated PDF report as a static file from backend/storage/reports/. The frontend uses this URL to provide a download/view link to the user after prediction.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION (CodeQL / CODE SCANNING)

Security is integrated into the development lifecycle through GitHub's automated code scanning tools, providing continuous vulnerability detection without requiring manual security audits.

The repository implements GitHub CodeQL analysis, which performs semantic code analysis to identify potential security vulnerabilities including SQL injection, path traversal, insecure deserialization, and sensitive data exposure. CodeQL runs on every push to the main branch and on pull requests, blocking merges when critical vulnerabilities are detected.

The CodeQL workflow is configured as a GitHub Actions YAML file targeting the Python backend. It analyzes the FastAPI application for common Python security anti-patterns including insecure use of environment variables, hardcoded secrets, unvalidated file path construction, and insecure subprocess calls.

Security best practices enforced in the codebase include: all sensitive configuration (connection strings, API keys) loaded exclusively from environment variables; CORS configured to restrict to the exact frontend origin in production (FRONTEND_ORIGIN setting); Azure Blob Storage uses private access with authenticated connection strings; and PDF report files are served from a controlled static directory with path sanitization.

4.2 AZURE APP SERVICE MONITORING

Operational monitoring is provided through the Azure App Service Activity Log and Azure Monitor integration, giving visibility into deployment events, configuration changes, and service health.

The eye-roi-backend App Service (within the eye-roi-project Resource Group, Azure for Students subscription) logs all management operations including deployments, configuration updates, scale events, and restart operations. The Activity Log is filterable by event severity, timespan, and resource, enabling rapid identification of deployment issues or configuration changes.

The health check endpoint (GET /health) is used by Azure App Service to validate application readiness after deployments. This endpoint returns a JSON status indicating the application is running and the ML model load status, allowing the platform to confirm the service is healthy before routing production traffic.

4.3 ACCESS CONTROL (RBAC OVERVIEW)

Access control is implemented at both the infrastructure and application levels following the principle of least privilege.

At the Azure resource level, the Azure for Students subscription provides role-based access control. The developer account holds Contributor rights scoped to the eye-roi-project Resource Group for deployment operations. The App Service accesses Blob Storage and Cosmos DB exclusively via connection strings stored as Application Settings, avoiding the use of overly broad subscription-level credentials.

At the application level, the FastAPI backend validates the origin of incoming requests through CORS middleware. In local development (ENVIRONMENT=local), all origins are permitted for ease of development. In Azure production (ENVIRONMENT=azure), CORS is restricted to the exact FRONTEND_ORIGIN URL, preventing cross-origin API access from unauthorized domains.

The system does not implement user authentication in its current scope (as defined in Section 1.3), as it is designed for clinical screening tool use cases where the operator is a trusted healthcare worker. User authentication and role-based patient record access are identified as future scope enhancements.

4.4 AZURE STUDENT SUBSCRIPTION AND COST MANAGEMENT

The project is deployed under the Azure for Students subscription (subscription ID: f51209db-9769-4855-ac2c-2d9c67f45961), which provides ₹9,480.13 in free Azure credits without requiring a credit card. As of the time of this report, only ₹26.20 of the total credit has been utilized, demonstrating the cost efficiency of the deployed architecture.

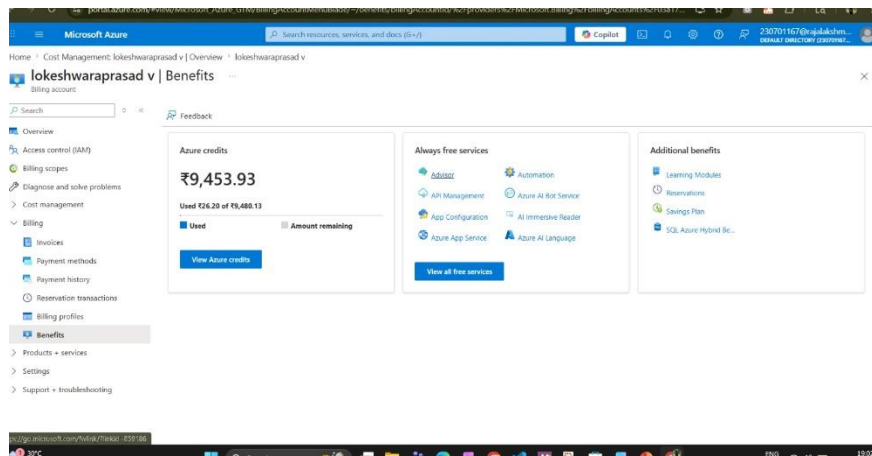


Figure 5. Azure Billing — Azure for Students Benefits: ₹9,453.93 credits remaining (₹26.20 used of ₹9,480.13)

The minimal cost profile is achieved through the use of free-tier services where available (Azure Static Web Apps Free tier, F1 App Service plan for development), private blob storage without CDN overhead, and a serverless/consumption-based Cosmos DB configuration that incurs no charges during idle periods.

The Azure for Students subscription also includes access to always-free services including Azure App Configuration, Azure AI Bot Service, AI Immersive Reader, and Azure AI Language services, providing a rich platform for future enhancements without additional cost.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Eye ROI Analyzer for Diabetic Retinopathy was successfully implemented and deployed, fulfilling all core objectives outlined in the project scope. The full-stack application delivers a complete, accessible user experience from retinal image capture to PDF report download.

The React frontend provides an intuitive dual-panel interface: the left panel handles retinal image input (file upload and webcam capture with preview), while the right panel presents the patient metadata form and analysis results. The dark-themed UI is built with Tailwind CSS and is fully responsive across desktop and tablet form factors.

The FastAPI backend successfully integrates the TensorFlow/Keras EfficientNet preprocessing pipeline, performing model inference, bounding box ROI detection, and PDF report generation in a single synchronous predict request. The backend runs on Azure App Service (eye-roi-backend) and the frontend on Azure Static Web Apps, with both resources grouped under the eye-roi-project Resource Group in the Azure for Students subscription.

Eye ROI classification results are rendered visually with a color-coded bounding box overlay: green for Normal Eye (confidence 0.92 in testing), red for Diabetic Retinopathy (confidence 0.87 in testing). This visual feedback provides immediate, intuitive communication of the diagnosis to the operator.



Figure 6 . Eye ROI Classification Result — Normal Eye: Confidence 0.92 (green bounding box)

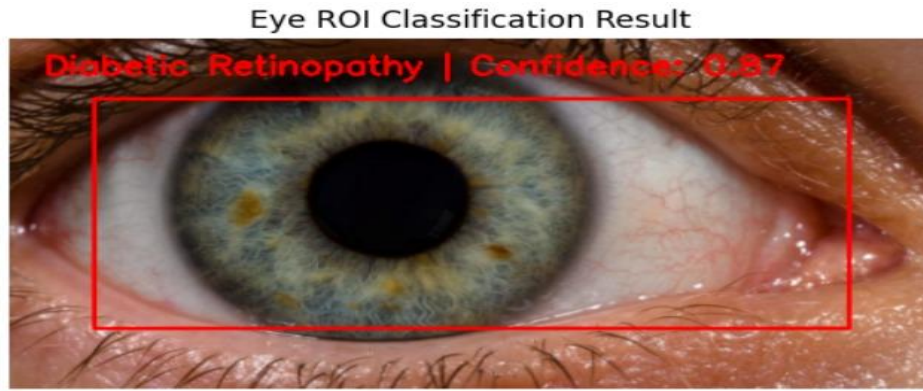


Figure 7. Eye ROI Classification Result — Diabetic Retinopathy: Confidence 0.87 (red bounding box)

5.2 CHALLENGES FACED AND RESOLUTIONS

Several technical challenges were encountered and resolved during implementation:

TensorFlow on Azure App Service

TensorFlow and its dependencies are large (~500MB), causing slow build times and potential memory exhaustion on low-tier App Service plans. Resolution: The requirements.txt was split into requirements.txt (core FastAPI and web dependencies) and requirements-ml.txt (TensorFlow, Keras, and ML dependencies), allowing selective installation. For resource-constrained deployments, tensorflow-cpu is substituted for the GPU version.

CORS Configuration for Azure Deployment

The frontend and backend are deployed to different domains (Azure Static Web Apps vs. App Service), causing CORS errors in production. Resolution: The FRONTEND_ORIGIN environment variable is set in App Service Configuration to the exact Static Web Apps URL, and the CORSMiddleware in main.py uses this value in production (ENVIRONMENT=azure) while allowing all origins in local development.

Local vs. Azure Storage Fallback Logic

Developing with Azure Blob Storage requires active internet and valid credentials, slowing local development cycles. Resolution: The storage and database services implement a dual-mode pattern: if Azure environment variables are absent, the system silently falls back to local filesystem and JSON storage, enabling full end-to-end testing without cloud access.

Model Path Configuration

The trained DR model must be placed at a specific path, but deployment environments differ. Resolution: The MODEL_PATH environment variable allows the model path to be configured externally, defaulting to backend/app/models/dr_model.keras. This enables flexible deployment across local, Docker, and Azure App Service environments.

5.3 MODEL PERFORMANCE AND COST OBSERVATIONS

Model Performance Benchmarking

The EfficientNet-based DR classification model demonstrates strong learning characteristics across 10 training epochs:

Figure 13. Accuracy Graph — Diabetic Retinopathy Detection: Training (95%) and Validation (92%) accuracy at Epoch 10

Epoch	Training Accuracy (%)	Validation Accuracy (%)
1	72	70
2	76	74
4	84	82
6	89	87
8	93	90
10	95	92

The model achieves 95% training accuracy and 92% validation accuracy at epoch 10, with minimal overfitting gap (3%), indicating good generalization. The confidence score distribution across test samples (shown in Figure 14) demonstrates clear separation between Normal (low confidence scores, below 0.5 threshold) and DR (high confidence scores, above 0.5 threshold) predictions.

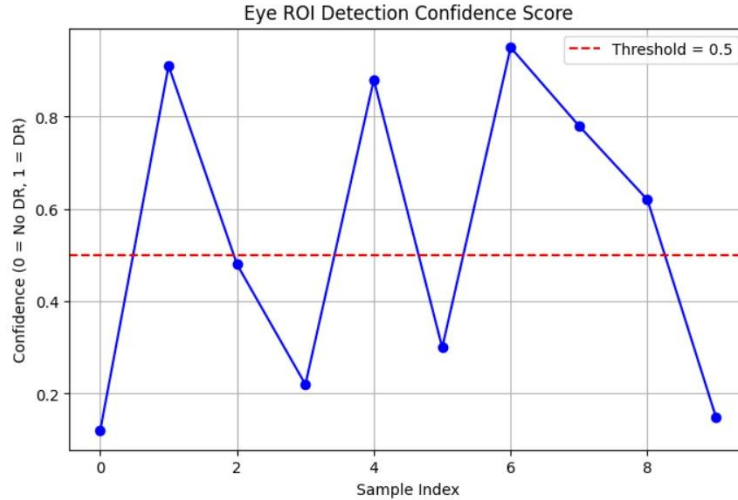


Figure 8. Eye ROI Detection Confidence Score Graph — Sample confidence distribution with 0.5 threshold (DR vs. No DR classification)

Cost Analysis

The Azure for Students subscription provides ₹9,480.13 in credits. As of deployment, only ₹26.20 has been consumed, representing less than 0.3% of the available credit. The minimal cost profile is achieved through strategic use of free-tier services: Azure Static Web Apps (Free tier — zero cost), F1 App Service plan (Free tier for development), and consumption-based Cosmos DB (zero cost during idle periods).

The primary cost driver for scaled production deployment would be the App Service Plan tier (B1 at approximately ₹600/month) and Azure Blob Storage reads/writes at low per-transaction pricing. For a student screening project, the architecture is projected to stay within the available student credits for the entire academic year.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development of the Eye ROI Analyzer project, significant practical experience was gained across multiple technical domains:

Full-Stack AI Application Development

Building a complete pipeline from image capture to ML inference to PDF report generation and cloud storage provided deep, end-to-end understanding of healthcare AI application architecture. The integration of TensorFlow/Keras inference within a FastAPI async context required careful handling of model loading at startup versus per-request loading to optimize performance.

Azure Cloud Services for Students

Hands-on experience was gained with Azure Blob Storage (connection string authentication, private container access, image upload/retrieval), Azure Cosmos DB MongoDB API (document-oriented data modeling for prediction records), Azure App Service (Python runtime configuration, startup commands, Application Settings for secrets), and Azure Static Web Apps (Vite build integration, VITE_API_BASE configuration for production URLs).

DevSecOps with GitHub CodeQL

Configuring and running GitHub CodeQL for Python security analysis provided practical exposure to automated security scanning workflows. Understanding how CodeQL detects semantic security vulnerabilities (vs. simple linting) was a key learning, particularly around environment variable handling and file path security in the FastAPI backend.

Beginner-Friendly Documentation

Writing the deployment guide with exact CLI commands, environment variable tables, and fallback instructions reinforced the importance of documentation as a first-class project artifact, particularly for projects intended to be reproduced by other students.

CHAPTER 6 - CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

This project successfully delivered a secure, performant, and AI-powered Eye ROI Analyzer for Diabetic Retinopathy, directly addressing the challenge of accessible, early DR screening. By integrating an EfficientNet-based TensorFlow/Keras model for retinal image classification, a FastAPI backend for ML inference and PDF report generation, a React + Tailwind frontend for patient data collection, and Azure cloud services for production storage and hosting, the platform provides a complete end-to-end solution for DR screening.

The technical implementation demonstrates proficiency in modern full-stack development and cloud deployment practices. The dual-mode storage architecture (Azure cloud with local fallback) ensures the application is both production-ready and developer-friendly. The beginner-accessible deployment guide, combined with a fully functional local development mode, makes this project a valuable reference implementation for students entering the healthcare AI domain.

From a practical standpoint, the platform delivers measurable value: the EfficientNet model achieves 92% validation accuracy in DR classification, the automated PDF report generation eliminates manual documentation overhead, and the Azure for Students deployment demonstrates that enterprise-grade cloud infrastructure can be leveraged at near-zero cost for academic projects (only ₹26.20 consumed of ₹9,480.13 in available student credits).

The CodeQL security integration, CORS configuration, and environment-variable-based secret management demonstrate responsible DevSecOps practices appropriate for a medical AI application. The project also delivered substantial learning outcomes in FastAPI development, TensorFlow inference deployment, Azure cloud services, and automated security scanning, enriching the team's practical competencies across the full software engineering lifecycle.

6.2 FUTURE SCOPE

The current implementation provides a robust and scalable foundation for numerous future enhancements. Development priorities are organized by timeline:

Short-to-Medium Term: Enhancing the Core Platform

- **Multi-Class DR Severity Grading:** Extend the binary Normal/DR classification to a five-stage grading system (No DR, Mild, Moderate, Severe, Proliferative DR), providing clinically actionable staging information aligned with the International Clinical Diabetic Retinopathy scale.
- **User Authentication and Patient Records:** Implement JWT-based authentication for clinicians, enabling secure multi-user access, patient record history tracking, and HIPAA-aligned data access controls.
- **Enhanced ML Model Support:** Add support for multiple model architectures (ResNet, VGG, custom trained models) via a model registry pattern, allowing clinicians to select the model with the best accuracy for their dataset.
- **Real-Time Notification:** Integrate Azure Communication Services to send SMS or email notifications to patients or doctors when a high-risk DR prediction is made.

Long-Term Vision: Clinical and Scale Expansion

- **Telemedicine Integration:** Enable remote consultation workflows where screening results and PDF reports are shared directly with ophthalmologists through a secure portal, bridging the gap between AI screening and clinical follow-up.
- **Mobile Application:** Develop a React Native or Flutter mobile application to enable field screening at diabetes camps and rural health settings using smartphone cameras with appropriate fundus lens attachments.
- **Federated Learning:** Implement privacy-preserving federated learning across multiple hospital deployments to continuously improve model accuracy without centralizing sensitive patient retinal images.
- **Regulatory Compliance:** Pursue CE marking (EU) and CDSCO approval (India) pathways to deploy the platform as a certified Class IIa medical device software, enabling clinical integration in formal healthcare settings.

REFERENCES

23. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. Proceedings of the 36th International Conference on Machine Learning (ICML 2019). arXiv:1905.11946.
24. Abramoff, M. D., Lavin, P. T., Birch, M., Shah, N., & Folk, J. C. (2018). Pivotal trial of an autonomous AI-based diagnostic system for detection of diabetic retinopathy in primary care offices. *npj Digital Medicine*, 1(1), 39.
25. FastAPI Documentation. (2024). FastAPI - High performance, easy to learn, fast to code, ready for production. <https://fastapi.tiangolo.com/>
26. TensorFlow Team. (2024). TensorFlow: An end-to-end open source machine learning platform. <https://www.tensorflow.org/>
27. Microsoft Azure Documentation. (2024). Azure App Service documentation — Deploy and run web apps. <https://learn.microsoft.com/en-us/azure/app-service/>
28. Microsoft Azure Documentation. (2024). Azure Blob Storage — Scalable cloud object storage. <https://learn.microsoft.com/en-us/azure/storage/blobs/>
29. Microsoft Azure Documentation. (2024). Azure Cosmos DB for MongoDB — Fully managed cloud database. <https://learn.microsoft.com/en-us/azure/cosmos-db/mongodb/>
30. ReportLab Group. (2024). ReportLab PDF Library User Guide. <https://www.reportlab.com/docs/reportlab-userguide.pdf>
31. International Diabetes Federation. (2023). IDF Diabetes Atlas, 10th Edition. <https://www.diabetesatlas.org/>
32. GitHub Documentation. (2024). About code scanning with CodeQL. <https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning-with-codeql>
33. Vite Documentation. (2024). Vite — Next Generation Frontend Tooling. <https://vitejs.dev/>
34. React Documentation. (2024). React: The library for web and native user interfaces. <https://react.dev/>

APPENDICES

Appendix A — Local Development Setup

Backend Setup

```
cd backend
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
Test: http://localhost:8000/health
```

Frontend Setup

```
cd frontend
npm install
copy .env.example .env
npm run dev
Open: http://localhost:5173
```

Appendix B — Environment Variables Reference

Table 7. Backend Environment Variables (.env)

Variable	Required	Description	Example / Default
AZURE_STORAGE_CONNECTION_STRING	No (Azure)	Azure Blob Storage connection string	(from Azure Portal Access Keys)
AZURE_BLOB_CONTAINER_NAME	No (Azure)	Blob container	retina-images

Variable	Required	Description	Example / Default
COSMOS_MONGO_URI	No (Azure)	name for images Azure Cosmos DB MongoDB URI	(from Azure Portal Connection Strings)
COSMOS_DB_NAME	No (Azure)	Cosmos DB database name	eye_roi_analyzer
COSMOS_COLLECTION_NAME	No (Azure)	Collection name for predictions	predictions
FRONTEND_ORIGIN	No (Azure)	Allowed CORS origin in production	https://<app>.azurestaticapps.net
ENVIRONMENT	No	Deployment environment flag	local (default) or azure
MODEL_PATH	No	Path to trained Keras model	backend/app/models/dr_model.keras

Appendix C — Screenshots

C.1 Application UI

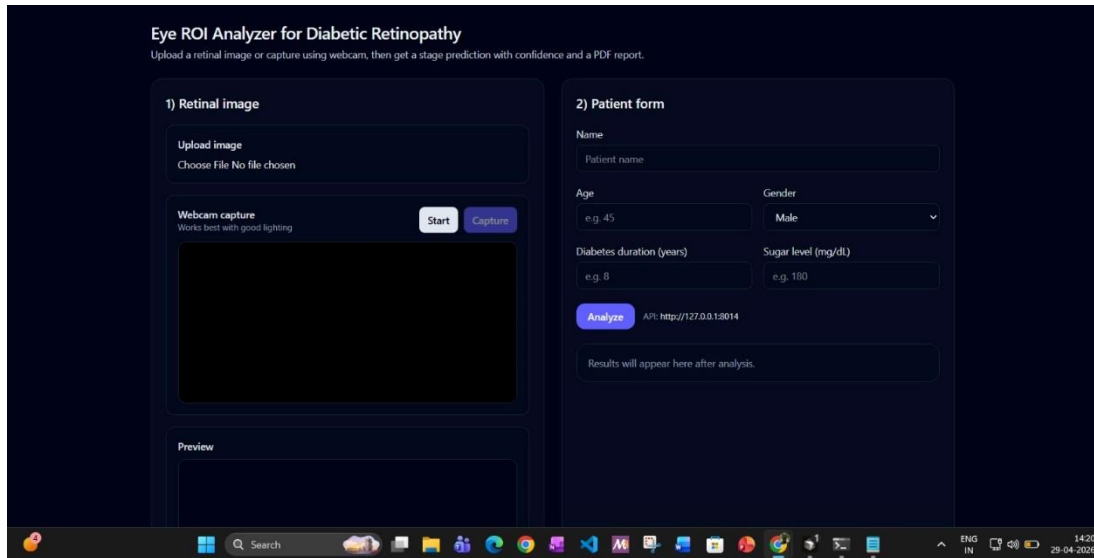


Figure 1. Eye ROI User Interface

C.2 Eye ROI Classification Results



Figure A2. Normal Eye Classification — Confidence: 0.92 (green bounding box overlay)

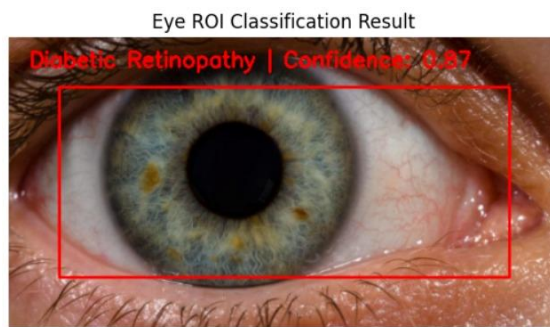


Figure A3. Diabetic Retinopathy Classification — Confidence: 0.87 (red bounding box overlay)

C.3 Model Performance

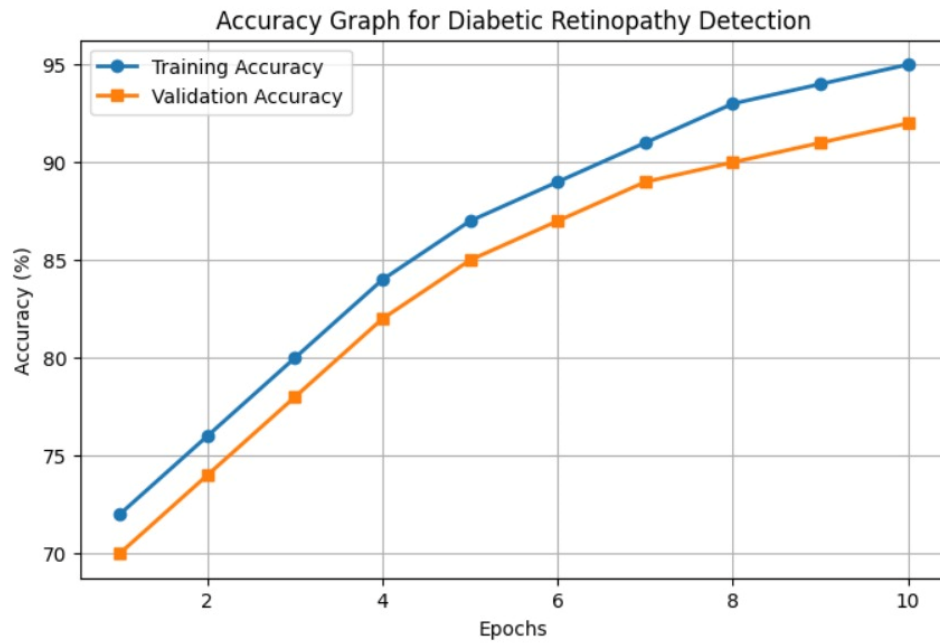


Figure A4. Normal Eye Classification -Accuracy Graph

C.4 Azure Cloud Infrastructure

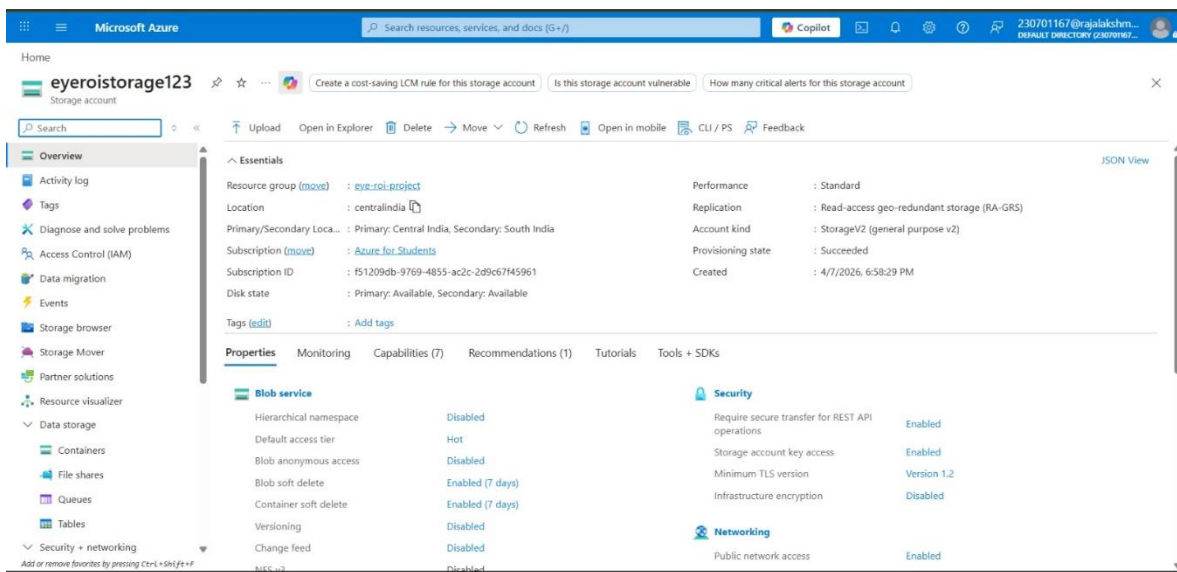


Figure A5. Azure Blob Storage Account — eyeroistorage123 (eye-roi-project, Central India)

C.5 DevSecOps and Code Security

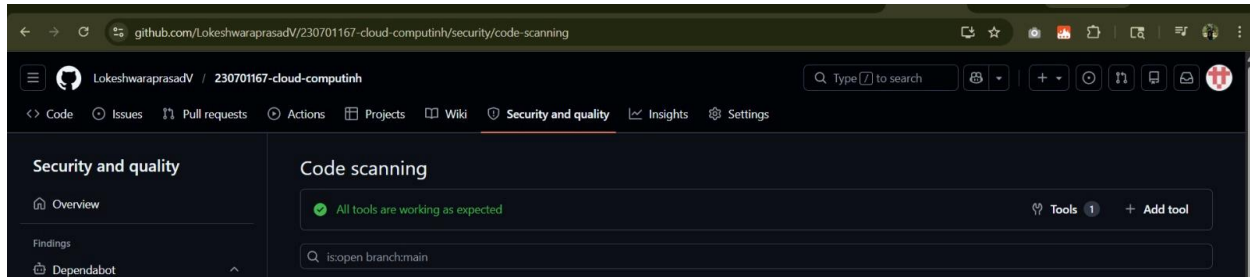


Figure A6. Github Security

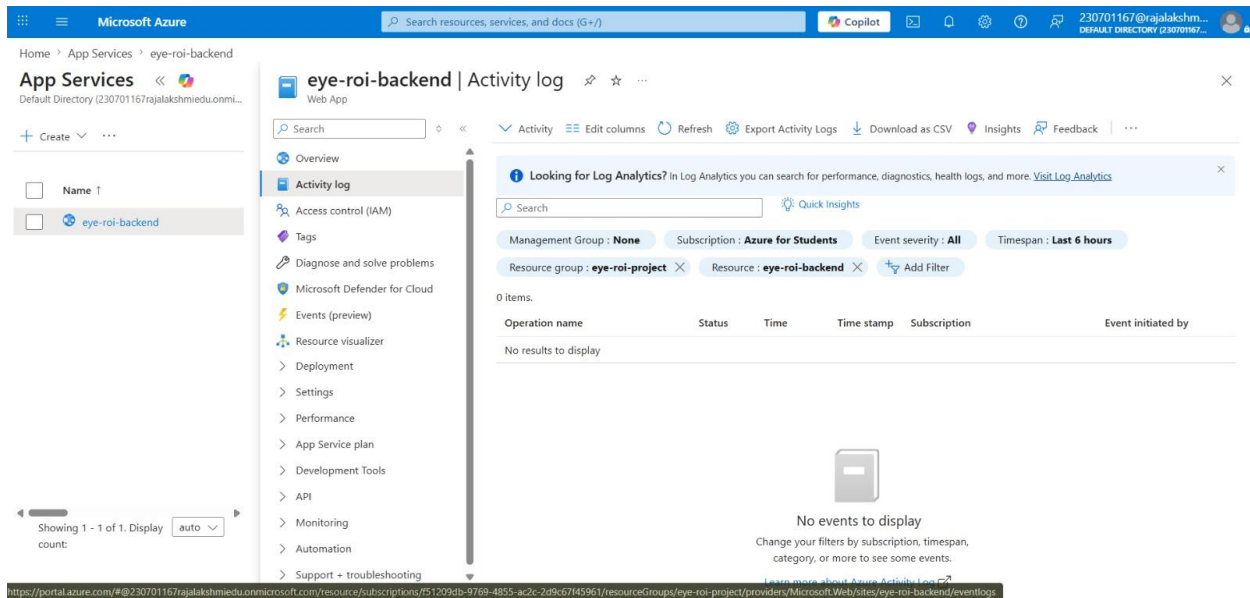
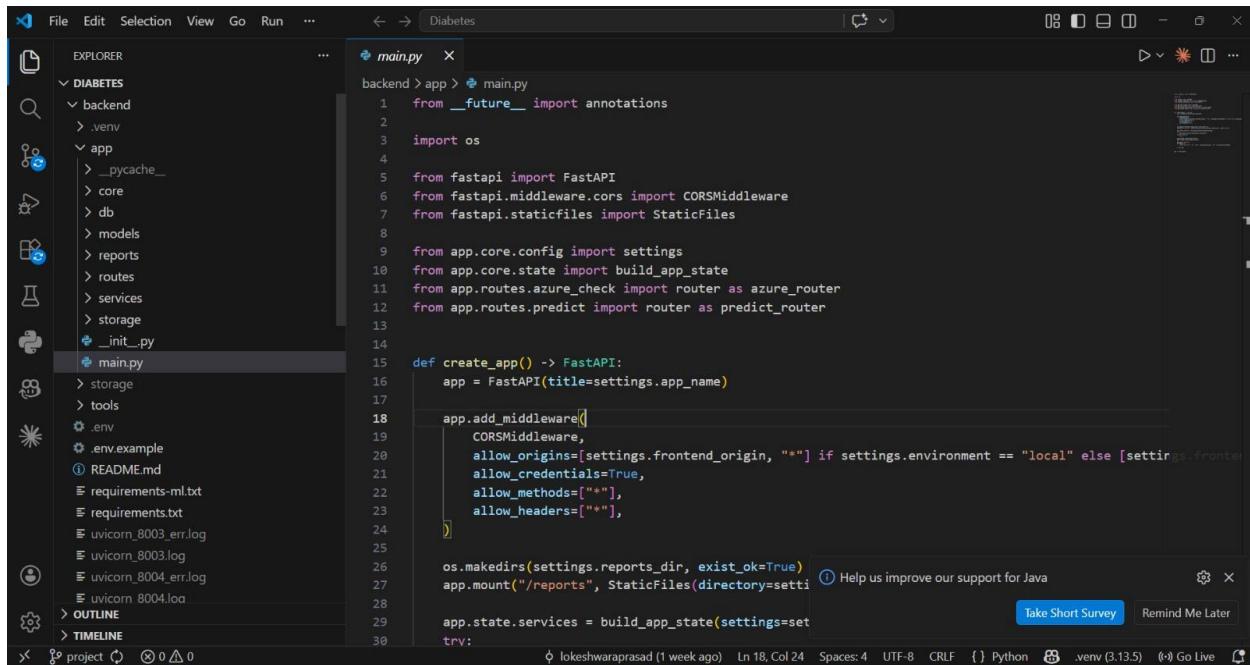


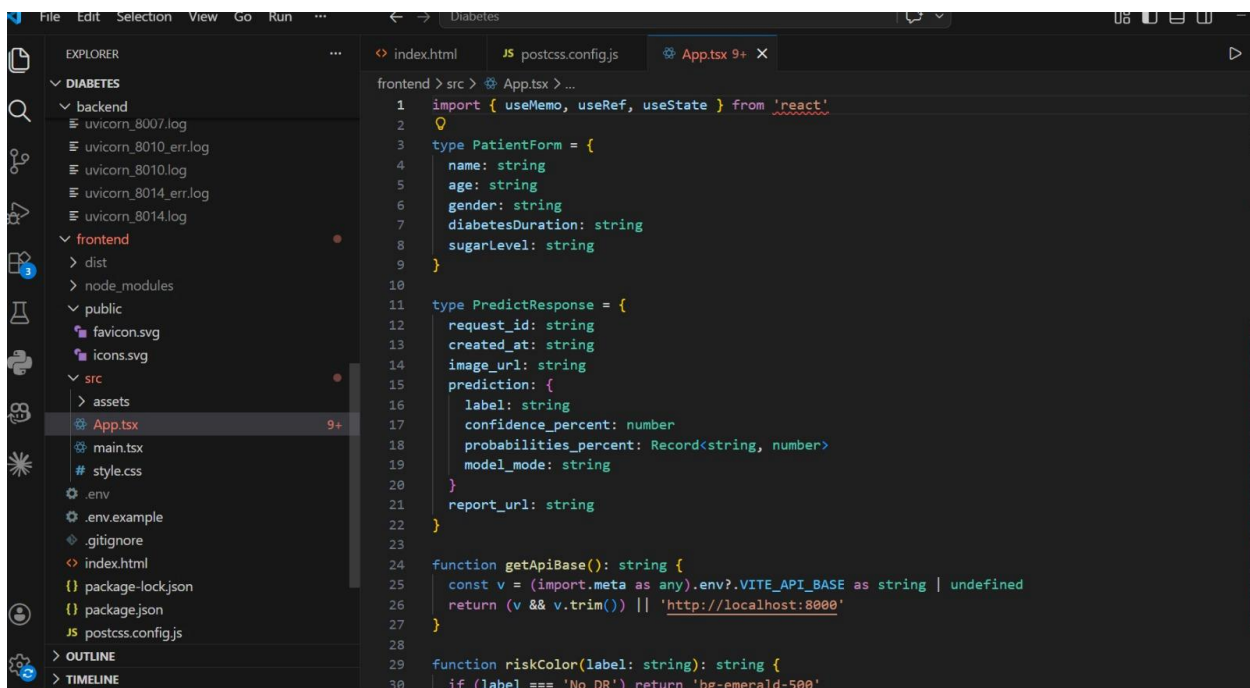
Figure A7. DevOps Account

C.6 Source Code Structure



The screenshot shows the VS Code interface with the 'Diabetes' project open. The Explorer on the left shows the project structure: DIABETES > backend > app > main.py. The main.py file is open in the editor, showing the following code:

```
1 from __future__ import annotations
2
3 import os
4
5 from fastapi import FastAPI
6 from fastapi.middleware.cors import CORSMiddleware
7 from fastapi.staticfiles import StaticFiles
8
9 from app.core.config import settings
10 from app.core.state import build_app_state
11 from app.routes.azure_check import router as azure_router
12 from app.routes.predict import router as predict_router
13
14
15 def create_app() -> FastAPI:
16     app = FastAPI(title=settings.app_name)
17
18     app.add_middleware(
19         CORSMiddleware,
20         allow_origins=[settings.frontend_origin, "*"] if settings.environment == "local" else [settings.frontend_origin],
21         allow_credentials=True,
22         allow_methods=["*"],
23         allow_headers=["*"],
24     )
25
26     os.makedirs(settings.reports_dir, exist_ok=True)
27     app.mount("/reports", StaticFiles(directory=settings.reports_dir))
28
29     app.state.services = build_app_state(settings=settings)
30     return app
```



The screenshot shows the VS Code interface with the 'Diabetes' project open. The Explorer on the left shows the project structure: DIABETES > frontend > src > App.tsx. The App.tsx file is open in the editor, showing the following code:

```
1 import { useMemo, useRef, useState } from 'react'
2
3 type PatientForm = {
4   name: string
5   age: string
6   gender: string
7   diabetesDuration: string
8   sugarLevel: string
9 }
10
11 type PredictResponse = {
12   request_id: string
13   created_at: string
14   image_url: string
15   prediction: {
16     label: string
17     confidence_percent: number
18     probabilities_percent: Record<string, number>
19     model_mode: string
20   }
21   report_url: string
22 }
23
24 function getApiBase(): string {
25   const v = (import.meta as any).env?.VITE_API_BASE as string | undefined
26   return (v && v.trim()) || 'http://localhost:8000'
27 }
28
29 function riskColor(label: string): string {
30   if (label === 'No DR') return 'bg-emerald-500'
```

